

Docker Basics, Dockerfile for Spring Boot, and Docker Compose

1. Docker Basics

1.1 What is Docker?

Docker is a tool that packages an application together with everything it needs to run — code, runtime, libraries, settings — into a single unit called a container. That container runs the same way on your laptop, a teammate's laptop, or a production server, because it carries its own environment with it.

Easy way to think about it: A container is like a sealed lunchbox. Whatever is packed inside travels with it and tastes the same wherever you open it — your machine doesn't need to already have the right ingredients.

1.2 Key Concepts

- **Image** — a snapshot of an environment plus your app code (read-only). Think of it as a recipe or a frozen template.
- **Container** — a running instance of an image. Think of it as the actual dish made from the recipe — you can start, stop, and remove it.
- **Dockerfile** — a text file with step-by-step instructions Docker follows to build an image.
- **Docker Hub / Registry** — an online library of pre-built images (e.g., official java, postgres, kafka images) that you can pull and use.
- **Docker Compose** — a tool to define and run multiple containers together using one YAML file (covered in Section 3).

1.3 Image vs Container — Easy Analogy

Image	Container
The blueprint / class	The actual running object / instance
Static, doesn't change	Has a live process, can change while running
Built once, reused many times	Many containers can be started from one image

1.4 Installing Docker

1. Download Docker Desktop from docker.com for Windows/Mac, or install docker engine via package manager on Linux
2. Start Docker Desktop (or the Docker daemon on Linux)

3. Confirm installation by checking the version

```
docker --version
```

1.5 Core Docker Commands — Syntax Explained

Pull an image from Docker Hub:

```
docker pull nginx
```

- **docker pull <image>** — downloads the image named nginx from Docker Hub onto your machine (doesn't run it yet)

Run a container from an image:

```
docker run -d -p 8080:80 --name my-nginx nginx
```

- **docker run** — creates and starts a container
- **-d** — “detached” — runs in the background instead of taking over your terminal
- **-p 8080:80** — maps port 8080 on your machine to port 80 inside the container (host:container)
- **--name my-nginx** — gives the container a friendly name instead of a random one

List running containers:

```
docker ps
```

- shows only currently running containers; add **-a** to see all containers including stopped ones (docker ps -a)

Stop and remove a container:

```
docker stop my-nginx  
docker rm my-nginx
```

- **docker stop <name>** — stops the running container (sends a shutdown signal)
- **docker rm <name>** — permanently deletes the stopped container (the image is untouched)

View logs from a container:

```
docker logs -f my-nginx
```

- **-f** — “follow” — keeps streaming new log lines live, similar to tail -f

List and remove images:

```
docker images
```

```
docker rmi nginx
```

1.6 Step-by-Step Demo: Run Your First Container

1. Pull and run an nginx web server

```
docker run -d -p 8080:80 --name demo-web nginx
```

2. Open a browser and go to <http://localhost:8080> — you'll see the default nginx welcome page
3. Check it's running

```
docker ps
```

4. Stop and clean up

```
docker stop demo-web
docker rm demo-web
```

2. Dockerfile for Spring Boot

2.1 What is a Dockerfile?

A Dockerfile is a plain text file containing a list of instructions. Docker reads it top to bottom and builds an image, one layer at a time. For a Spring Boot app, the Dockerfile typically: takes a Java base image, copies in the built JAR file, and tells Docker how to start the app.

2.2 Simple Dockerfile (Single-Stage)

Assuming you've already run `mvn clean package` locally and have a JAR in the `target/` folder:

```
# Step 1: Start from a lightweight Java 17 runtime image
FROM eclipse-temurin:17-jre-alpine

# Step 2: Set the working directory inside the container
WORKDIR /app

# Step 3: Copy the built JAR from your machine into the image
COPY target/*.jar app.jar

# Step 4: Document which port the app listens on
EXPOSE 8080

# Step 5: Command to run when the container starts
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Syntax explained, line by line

- **FROM** — sets the base image — the starting point for everything else. `eclipse-temurin:17-jre-alpine` is a small Linux image with just the Java 17 runtime (no full JDK, no extra tools) — keeps the final image small
- **WORKDIR /app** — sets the default folder inside the container where following commands will run; if it doesn't exist, Docker creates it
- **COPY target/*.jar app.jar** — copies a file from your local machine (left side) into the image (right side); here it renames whatever JAR is built to `app.jar` for simplicity
- **EXPOSE 8080** — documentation only — tells humans and tools which port the app uses; it does NOT actually publish the port (that happens with `-p` at docker run, or in Compose)
- **ENTRYPOINT [...]** — the command that runs when a container starts from this image; written in JSON-array form for cleaner process handling

2.3 Better Practice: Multi-Stage Dockerfile (Build + Run)

This approach builds the JAR inside Docker itself (so you don't need Maven installed locally) and produces a smaller final image because build tools are discarded after compiling.

```
# ----- Stage 1: Build -----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

# ----- Stage 2: Run -----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Syntax explained, line by line

- **FROM maven:3.9-eclipse-temurin-17 AS build** — starts the first stage using a Maven + Java image, and names this stage “build” so we can reference it later
- **COPY pom.xml . + RUN mvn dependency:go-offline** — copies only `pom.xml` first, then downloads dependencies (`mvn dependency:go-offline`) — this is a caching trick: if only your source code changes (not `pom.xml`), Docker reuses the cached dependency layer and skips re-downloading everything
- **COPY src ./src + RUN mvn clean package -DskipTests** — now copies the actual source code and builds the JAR, skipping tests for a faster image build
- **FROM eclipse-temurin:17-jre-alpine (again)** — starts a brand new, clean, lightweight stage — none of the Maven tooling from Stage 1 carries over, keeping the final image small
- **COPY --from=build /app/target/*.jar app.jar** — copies the JAR built in Stage 1 (named “build”) into this new, clean image

Why multi-stage matters: The final image only contains the JRE and your JAR — not Maven, not your source code, not test files. This results in a smaller, faster, more secure image to deploy.

2.4 Step-by-Step Demo: Build and Run a Spring Boot Image

1. Make sure you're in your Spring Boot project's root folder (where pom.xml lives)
2. Create a file named Dockerfile (no extension) with the multi-stage content above
3. Build the image

```
docker build -t my-springboot-app .
```

4. Run a container from that image

```
docker run -d -p 8080:8080 --name springboot-demo my-springboot-app
```

5. Test it

```
curl http://localhost:8080/api/orders
```

6. Check logs if something looks wrong

```
docker logs -f springboot-demo
```

Syntax explained

- **docker build -t my-springboot-app .** — docker build builds an image from the Dockerfile; -t my-springboot-app "tags" (names) the image; the trailing dot means "use the current folder as the build context"

3. Docker Compose

3.1 What is Docker Compose?

Real applications rarely run alone — a Spring Boot app might need Kafka, a database, and Zookeeper all running together. Instead of starting each container manually with long docker run commands, Docker Compose lets you describe all of them in a single YAML file and start everything with one command.

Easy way to think about it: Docker Compose is like a recipe card for a multi-course meal — instead of cooking each dish separately and timing them yourself, you describe them all in one place and Compose coordinates starting them together.

3.2 Basic Structure of docker-compose.yml

```
version: "3.8"

services:
```

```

service-name-1:
  image: some-image
  ports:
    - "hostPort:containerPort"
  environment:
    - KEY=value

service-name-2:
  build: .
  depends_on:
    - service-name-1

```

Syntax explained

- **version** — the Compose file format version (3.8 is widely supported and stable)
- **services** — the top-level list of containers you want to run; each key under it (like service-name-1) becomes one container
- **image** — use a ready-made image from Docker Hub instead of building one
- **build** — build the image from a Dockerfile in the given folder (. means current folder) instead of pulling one
- **ports** — maps host:container ports, same idea as docker run -p
- **environment** — sets environment variables inside the container
- **depends_on** — tells Compose to start this service after the listed ones (note: this controls start order, not full readiness — the app inside may still need a moment to be ready)

3.3 Full Example: Spring Boot + Kafka + Zookeeper

This brings together everything from the Dockerfile section with Kafka, all started with one command.

```

version: "3.8"

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

```

```
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

springboot-app:
  build: .
  depends_on:
    - kafka
  ports:
    - "8080:8080"
  environment:
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
```

Important detail: container-to-container networking

Notice that inside the kafka service, KAFKA_ADVERTISED_LISTENERS uses kafka:9092, not localhost:9092. Compose automatically creates a private network where each service can reach others using their service name as the hostname. So from inside springboot-app, the Kafka broker is reachable at kafka:9092 — not localhost, because localhost inside a container refers to that container itself, not its neighbors.

3.4 Core Docker Compose Commands

Start everything (build images if needed, then start all containers):

```
docker compose up -d
```

- **-d** — runs containers in the background (detached mode), same idea as docker run -d

Rebuild images before starting (useful after changing your Dockerfile or code):

```
docker compose up -d --build
```

View running services:

```
docker compose ps
```

View logs from all services, or one specific service:

```
docker compose logs -f
docker compose logs -f springboot-app
```

Stop and remove everything Compose created (containers, network):

```
docker compose down
```

3.5 Step-by-Step Demo: Bring Up the Full Stack

1. Place docker-compose.yml in your Spring Boot project root, next to the Dockerfile from Section 2
2. Start everything

```
docker compose up -d --build
```

3. Confirm all three services are running

```
docker compose ps
```

4. Watch the Spring Boot logs to confirm it connected to Kafka

```
docker compose logs -f springboot-app
```

5. Test the app the same way as before

```
curl -X POST http://localhost:8080/api/orders \
  -H "Content-Type: application/json" \
  -d '{"id":101,"customer":"Asha","amount":250}'
```

6. Shut everything down when finished

```
docker compose down
```

Troubleshooting Checklist

Problem	Likely Cause	Fix
Port already in use	Another process or container is bound to that port	Stop the other process, or change the host-side port mapping (e.g. "8081:8080")
App can't reach Kafka inside Compose	Using localhost instead of the service name	Use the Kafka service name (e.g. kafka:9092) as the host, not localhost
docker build fails on COPY target/*.jar	JAR not built yet (single-stage Dockerfile)	Run mvn clean package first, or switch to the multi-stage Dockerfile which builds it for you
Container exits immediately	App crashed on startup	Check docker logs <container> or docker compose logs -f for the stack trace
Changes to code not appearing	Old image is being reused	Rebuild with docker compose up -d --build or docker build again

Quick Recap

Concept	Purpose	Key Command / Syntax
Image	Blueprint for a container	docker pull / docker build
Container	Running instance of an image	docker run / docker ps
Dockerfile	Instructions to build an image	FROM, WORKDIR, COPY, ENTRYPOINT
Multi-stage build	Smaller, cleaner final image	FROM ... AS build, COPY --from=build
Docker Compose	Run multiple containers together	docker compose up -d / down
Service networking	Containers reach each other by service name	kafka:9092 instead of localhost:9092

Suggested Training Flow (Demo Order)

1. Run the basic nginx container demo to introduce images vs. containers
2. Build the Spring Boot Dockerfile (single-stage first, then show the multi-stage upgrade)
3. Run the Spring Boot image standalone with docker run and test with curl
4. Introduce Docker Compose by adding Kafka + Zookeeper alongside the app
5. Bring up the full stack with one command and re-run the same curl test to show it still works end-to-end